

Module 3: Data Structures - Tuples and Sets

Dr. Smith

Review: Control Flow and Data Structures

Last Week's Highlights

- **Control Flow:**
 - if, elif, else statements
 - for loops
 - while loops
- **Data Structures:**
 - Lists
 - Dictionaries

Example: Filtering Customers for a Promotion

```
customers = {  
    "C001": {"name": "Alice", "city": "New York", "spent": 120},  
    "C002": {"name": "Bob", "city": "Los Angeles", "spent": 80},  
    "C003": {"name": "Charlie", "city": "New York", "spent": 150}  
}  
  
for customer_id, data in customers.items():  
    if data["city"] == "New York" and data["spent"] > 100:  
        print(f"Promotion Offer for {data['name']} ({customer_id})")
```

Promotion Offer for Alice (C001)
Promotion Offer for Charlie (C003)

Today's Agenda

What We'll Cover

- **Tuples:**
 - What they are and why they're useful.
 - Creating and accessing tuples.
 - The concept of immutability.
- **Sets:**
 - What they are and how they differ from lists and tuples.

- Creating sets and performing operations (union, intersection, difference).
 - **Functions:**
 - What functions are and why we need them.
 - Defining and calling functions.
 - Understanding arguments and return values.
 - Variable Scope (Local vs. Global)
 - Lambda Functions
-

Tuples

Introduction to Tuples

- **What are they?** Ordered, immutable collections of items.
- **Why use them?**
 - **Data Integrity:** Protect data from accidental modification.
 - **Performance:** Slightly faster than lists in some cases.
 - **Dictionary Keys:** Can be used as keys in dictionaries (unlike lists).

Mutability vs. Immutability

- **Mutable Data Structures:**
 - **Lists and Dictionaries** are mutable.
 - **Mutable** means that the contents of the data structure can be changed after it's created. You can add, remove, or modify elements.

```
# Example with a list (mutable)
my_list = [1, 2, 3]
my_list.append(4) # Add an element
my_list[0] = 0    # Modify an element
print(my_list)

# Example with a dictionary (mutable)
my_dict = {"a": 1, "b": 2}
my_dict["c"] = 3 # Add a key-value pair
my_dict["a"] = 0 # Modify a value
print(my_dict)
```

```
[0, 2, 3, 4]
{'a': 0, 'b': 2, 'c': 3}
```

- **Immutable Data Structures:**

- **Tuples** are immutable.
- **Immutable** means that once the data structure is created, its contents **cannot** be changed. You cannot add, remove, or modify elements.

```
# Example with a tuple (immutable)
my_tuple = (1, 2, 3)
# my_tuple.append(4) # This would raise an AttributeError
# my_tuple[0] = 0    # This would raise a TypeError
print(my_tuple)
```

```
(1, 2, 3)
```

Creating Tuples

- Use parentheses () to create tuples.
- Elements can be of different data types.

```
# A tuple representing a product
product = ("Laptop", "P001", 999.99, True)

# A tuple of coordinates
coordinates = (10, 20)

# An empty tuple
empty_tuple = ()
```

Accessing Tuple Elements

- Access elements using their index (starting from 0).
- Similar to lists, but you cannot modify elements after creation.

```
product_name = product[0] # Access the first element ("Laptop")
product_price = product[2] # Access the third element (999.99)

print(product_name)
print(product_price)
```

Laptop
999.99

Immutability (in Detail)

- **Key Feature of Tuples:** Once created, you cannot change their contents (add, remove, or modify elements).
- **Why is this useful?**
 - Ensures data consistency.
 - Makes code more predictable and less prone to errors.
 - Allows tuples to be used as dictionary keys (because they are hashable).

```
# Trying to modify a tuple will result in an error
product[1] = "P002" # TypeError: 'tuple' object does not support item assignment
```

Tuple Packing and Unpacking

- **Packing:** Creating a tuple from individual values.
- **Unpacking:** Assigning tuple elements to individual variables.

```
# Packing
employee = ("John Doe", 35, "Sales Manager")

# Unpacking
name, age, position = employee

print(name)
print(age)
print(position)
```

John Doe
35
Sales Manager

Common Tuple Use Cases

- **Data Records:** Representing fixed collections of related data (e.g., customer information, product details).
- **Function Arguments and Return Values:** Passing multiple values to or from functions.
- **Data Integrity:** Ensuring data remains constant throughout a program.
- **Representing coordinates, dimensions, or other structured data**

Tuple Methods

Tuples have a few built-in methods:

- `count(x)`: Returns the number of times `x` appears in the tuple.
- `index(x)`: Returns the index of the first occurrence of `x`.

```
numbers = (1, 2, 3, 2, 4, 2)
count_of_twos = numbers.count(2) # 3
index_of_three = numbers.index(3) # 2

print(count_of_twos)
print(index_of_three)
```

3
2

Sets

What are Sets?

- **Unordered collections of unique elements.**
- **Key Features:**
 - No duplicate values allowed.
 - Highly efficient for checking membership (whether an element exists in the set).
 - Support set operations (union, intersection, difference).

Creating Sets

- Use curly braces {} or the `set()` function.

```
# Creating sets using curly braces
fruits = {"apple", "banana", "orange", "apple"} # Duplicates are automatically removed
print(fruits) # Note: Order may vary

# Creating a set from a list
colors = set(["red", "green", "blue", "red"])
print(colors)
```

```
{'apple', 'orange', 'banana'}
{'red', 'green', 'blue'}
```

Adding and Removing Elements

- `add(x)`: Adds element `x` to the set.
- `remove(x)`: Removes element `x` from the set (raises an error if `x` is not present).
- `discard(x)`: Removes element `x` if it's present (no error if not present).
- `pop()`: Removes and returns an arbitrary element.
- `clear()`: Removes all elements.

```
fruits.add("grape")
fruits.remove("banana")
fruits.discard("kiwi") # No error even if "kiwi" is not present

print(fruits)
```

```
{'apple', 'grape', 'orange'}
```

Set Operations

- **Union (|)**: Combines elements from both sets.
- **Intersection (&)**: Returns elements present in both sets.
- **Difference (-)**: Returns elements in the first set but not in the second.
- **Symmetric Difference (^)**: Returns elements that are in either set, but not both.

```

set1 = {1, 2, 3}
set2 = {3, 4, 5}

union_set = set1 | set2          # {1, 2, 3, 4, 5}
intersection_set = set1 & set2   # {3}
difference_set = set1 - set2     # {1, 2}
symmetric_difference_set = set1 ^ set2 # {1, 2, 4, 5}

print("set1:", set1, "\nset2:", set2)
print("Union:", union_set)
print("Intersection:", intersection_set)
print("Difference:", difference_set)
print("Sym. Diff.:", symmetric_difference_set)

```

```

set1: {1, 2, 3}
set2: {3, 4, 5}
Union: {1, 2, 3, 4, 5}
Intersection: {3}
Difference: {1, 2}
Sym. Diff.: {1, 2, 4, 5}

```

Common Set Use Cases

- Removing duplicates from a list.
- Checking for membership (e.g., is a customer in a list of VIPs?).
- Finding unique items (e.g., unique products purchased).
- Analyzing relationships between data sets (e.g., common customers between two marketing campaigns).

Functions

What are Functions?

- Reusable blocks of code that perform a specific task.
- Why use them?
 - **Organization:** Break down complex problems into smaller, manageable parts.
 - **Reusability:** Write once, use many times.

- **Readability:** Make your code easier to understand.
- **Maintainability:** Easier to update and fix.

Defining Functions

- Use the `def` keyword followed by the function name, parentheses `()`, and a colon `:`.
- Indent the function body (the code that belongs to the function).

```
def greet(name):  
    """This function greets the person passed in as a parameter."""  
    print(f"Hello, {name}!")
```

Calling Functions

- Use the function name followed by parentheses `()`.
- Pass any required arguments (inputs) inside the parentheses.

```
greet("Alice") # Output: Hello, Alice!  
greet("Bob")   # Output: Hello, Bob!
```

Hello, Alice!
Hello, Bob!

Arguments

- **Inputs to a function.**
- Placeholders for values that will be used inside the function.
- Can have multiple arguments separated by commas.

```
def add_numbers(x, y):  
    """This function adds two numbers."""  
    sum = x + y  
    print(f"The sum is: {sum}")  
  
add_numbers(5, 3) # Output: The sum is: 8
```

The sum is: 8

Return Values

- **Output of a function.**
- Use the `return` keyword to specify the value to be returned.
- A function can return only one value, but that value can be a collection (e.g., a list or tuple).

```
def multiply_numbers(x, y):  
    """This function multiplies two numbers and returns the result."""  
    result = x * y  
    return result  
  
product = multiply_numbers(4, 6)  
print(product)  # Output: 24
```

24

Default Argument Values

- Assign default values to arguments in the function definition.
- Makes the arguments optional when calling the function.

```
def greet(name, greeting="Hello"):  
    """Greet a person with an optional custom greeting."""  
    print(f"{greeting}, {name}!")  
  
greet("Alice")  # Output: Hello, Alice!  
greet("Bob", "Good morning") # Output: Good morning, Bob!
```

Hello, Alice!
Good morning, Bob!

Keyword Arguments

- Pass arguments using the argument name and the assignment operator `=`.
- Allows you to pass arguments in any order.

```
def describe_product(name, price, category="Electronics"):
    """Describes a product with its name, price, and category."""
    print(f"Product: {name}, Price: ${price}, Category: {category}")

describe_product(name="Laptop", price=999.99)
describe_product(price=29.99, name="Mouse", category="Accessories")
```

Product: Laptop, Price: \$999.99, Category: Electronics
 Product: Mouse, Price: \$29.99, Category: Accessories

Business Scenario: Calculating Discounts

Let's create a function to calculate the final price of an item after applying a discount:

```
def calculate_discounted_price(price, discount_percentage):
    """Calculates the final price after applying a discount."""
    discount_amount = price * (discount_percentage / 100)
    final_price = price - discount_amount
    return final_price

original_price = 100
discount = 10
discounted_price = calculate_discounted_price(original_price, discount)
print(f"The discounted price is: ${discounted_price:.2f}")
```

The discounted price is: \$90.00

Variable Scope: Local vs. Global

- **Scope:** The region of a program where a variable is accessible.
- **Local Variables:** Defined inside a function and only accessible within that function.
- **Global Variables:** Defined outside any function and accessible throughout the program.

Local Scope

- Variables defined inside a function have local scope.
- They are created when the function is called and destroyed when the function finishes.

```
def my_function():
    local_var = 10 # local_var has local scope
    print(local_var)

my_function()
# print(local_var) # Error: local_var is not defined outside the function
```

10

Global Scope

- Variables defined outside any function have global scope.
- They can be accessed from anywhere in the program, including inside functions.

```
global_var = 20 # global_var has global scope

def another_function():
    print(global_var) # Accessing the global variable

another_function()
print(global_var)
```

20

20

Modifying Global Variables from Inside a Function

- Use the `global` keyword to indicate that you want to modify a global variable from within a function.

```
counter = 0

def increment_counter():
    global counter # Declare counter as global
    counter += 1

increment_counter()
increment_counter()
print(counter) # Output: 2
```

2

Why is Scope Important?

- **Encapsulation:** Local scope helps keep functions self-contained and prevents unintended side effects.
- **Readability:** Makes it easier to understand where variables are used and modified.
- **Maintainability:** Reduces the risk of naming conflicts and makes code easier to debug.

Lambda Functions

- **Anonymous functions:** Small, single-line functions that don't need a formal `def` statement.
- **Syntax:** `lambda arguments: expression`
- **Use Cases:**
 - Short, simple operations that you don't want to define as separate functions.
 - Often used with functions like `map()`, `filter()`, and `sorted()`.

Lambda Function Examples

```
# A lambda function to square a number
square = lambda x: x * x
print(square(5)) # Output: 25

# A lambda function to add two numbers
add = lambda x, y: x + y
print(add(3, 7)) # Output: 10

# A lambda function to check if a number is even
is_even = lambda x: x % 2 == 0
print(is_even(4)) # Output: True
print(is_even(7)) # Output: False
```

```
25
10
True
False
```

Using Lambda Functions with `map()`

- `map()` applies a function to each item in an iterable (e.g., a list).
- Lambda functions are often used as the function to be applied.

```
numbers = [1, 2, 3, 4, 5]

# Square each number in the list using a lambda function and map()
squared_numbers = list(map(lambda x: x * x, numbers))

print(squared_numbers) # Output: [1, 4, 9, 16, 25]
```

```
[1, 4, 9, 16, 25]
```

Using Lambda Functions with `filter()`

- `filter()` creates a new iterable with elements that satisfy a given condition.
- Lambda functions are often used to define the condition.

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

# Get only the even numbers using a lambda function and filter()
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))

print(even_numbers) # Output: [2, 4, 6, 8, 10]
```

```
[2, 4, 6, 8, 10]
```

Your Turn!

Practice Exercises

Open a new notebook in Google Colab and try these exercises.

Exercise 1: Tuple Creation and Access

- Create a tuple named `customer` with the following information:
 - Customer ID: “C0
 - Name: “Alice Smith”
 - Email: “alice.smith@email.com”
 - Is a loyalty member: True
- Access and print the customer’s name and email.

Exercise 2: Set Operations

- Create two sets:
 - `products_a`: {“Laptop”, “Mouse”, “Keyboard”}
 - `products_b`: {“Monitor”, “Keyboard”, “Webcam”}
- Find the following:
 - The union of `products_a` and `products_b`.
 - The intersection of `products_a` and `products_b`.
 - The products that are in `products_a` but not in `products_b`.

Exercise 3: Function Definition

- Write a function called `calculate_total_cost` that takes two arguments:
 - `price`: The price of an item.
 - `quantity`: The quantity of the item.
- The function should return the total cost (`price * quantity`).
- Test your function with different values for price and quantity.

Exercise 4: Scope

- Analyze the following code and predict the output:

```
x = 10

def my_function():
    x = 5
    print(f"Inside function: x = {x}")
```

```
my_function()
print(f"Outside function: x = {x}")
```

Inside function: x = 5
Outside function: x = 10

Exercise 5: Lambda Function

- Write a lambda function that takes a string as input and returns the string in uppercase.
- Test your lambda function with different strings.

Conclusion

- You've learned about tuples, sets, and functions - essential building blocks for more complex Python programs.
- We explored concepts like immutability, set operations, function arguments, return values, variable scope, and lambda functions.
- These tools will be crucial as we move on to working with external libraries like NumPy and Pandas for data analysis.

Next Week

- **Introduction to Libraries:** We'll start exploring the power of external libraries.
- **NumPy:** Get ready to work with numerical data efficiently using NumPy arrays and array operations.

Tasks for This Week

- Practice, practice, practice
 - learning to program is like learning a language, or playing a sport
 - * you can NOT cram the night before you need the skill!
 - * It's not hard if you do a little everyday - make it a habit
 - Practice using the problem set for this week.
 - * Try to complete each question before looking at the solution
 - Join extra help sessions on Saturday's (1 to 4pm)

- Complete **Assignment03**
 - Simple assignment to help you get started
- Prepare for **Quiz02** next class
 - In-class quiz covering the content from this week (Week03)

Questions?

- Don't hesitate to ask questions during class, office hours, or by email.
- The more you practice and engage with the material, the better you'll understand it.
- Remember, learning to code takes time and effort. Be patient with yourself, celebrate your progress, and keep practicing!